```
    function bid(bytes32 blindBid) public payable validPhase(Phase.Bidding)
    {                                                                    ❷
        bids[msg.sender] = Bid({
            blindedBid: blindBid,
            deposit: msg.value
        });
    }

    function reveal(uint value, bytes32 secret) public                  ❸
            validPhase(Phase.Reveal)
    {
        uint refund = 0;
            Bid storage bidToCheck = bids[msg.sender];
            if (bidToCheck.blindedBid == keccak256(abi.encodePacked(value,
                    secret)))
            {
            refund += bidToCheck.deposit;
            if (bidToCheck.deposit >= value) {
                if (placeBid(msg.sender, value))
                    refund -= value;
            }}

        msg.sender.transfer(refund);
    }

    function placeBid(address bidder, uint value) internal               ❹
            returns (bool success)
    {
        if (value <= highestBid) {
            return false;
        }
        if (highestBidder != address(0)) {
            // Refund the previously highest bidder
            depositReturns[highestBidder] += highestBid;
        }
        highestBid = value;
        highestBidder = bidder;
        return true;
    }

    // Withdraw a non-winning bid
    function withdraw() public {                                         ❺

        uint amount = depositReturns[msg.sender];
        require (amount > 0);
        depositReturns[msg.sender] = 0;
        msg.sender.transfer(amount);
        }
    }

    // End the auction and send the highest bid to the beneficiary       ❻
    function auctionEnd() public  validPhase(Phase.Done)
    {
        beneficiary.transfer(highestBid);
    }
}
```

❶ Constructor sets the beneficiary

❷ Blind bid function

❸ reveal() function checks blind bid